Safari Lanka

Design Patterns - Implementation, Code, Ranking & Viva Guide

Web Based Wildlife Safari Trip Management System

What changed in this build

A real GoF Strategy Pattern is now implemented for payments. Card and bank-transfer behavior has been moved into separate concrete strategy classes and selected through a PaymentContext at runtime. Existing payment rules and workflow are preserved.

Based on: IT2020 - Software Engineering, Lab 06 - Design Patterns (Strategy Pattern), plus the actual Safari Lanka source code.

Report scope: Strategy, Repository, Dependency Injection, Observer, MVC/Layered Architecture, Provider/Context, Service Layer, State-based Workflow, and Spring singleton-scoped bean behavior.

Strongest GoF pattern Strategy Pattern	**Best viva example** Card vs Bank Transfer
Backend support patterns Repository + Dependency Injection	**Frontend example** Observer in LanguageContext

Prepared as a colorful project report suitable for screenshots, report submission, and viva preparation.

1. Lab 06 Alignment

The uploaded Lab 06 describes the **Strategy Pattern** as a way to define a family of behaviors, place each behavior in a separate class, and let a context use or switch those behaviors without containing all implementation details itself.

Lab 06 element	Safari Lanka implementation
Strategy interface	**PaymentStrategy**
Concrete strategies	**CardPaymentStrategy**, **BankTransferPaymentStrategy**
Context class	**PaymentContext**
Runtime behavior selection	PaymentMethod.CARD or PaymentMethod.BANK_TRANSFER
Client / caller	PaymentController delegates payment processing

2. Pattern Strength Ranking

Rank	Pattern	Strength	Why
1	**Strategy**	**5/5 - STRONGEST**	Explicit GoF interface + concrete strategies + context; directly matches Lab 06.
2	**Repository**	**5/5 - STRONG**	Clear Spring Data repository abstraction across entities.
3	**Dependency Injection**	**5/5 - STRONG**	Constructor injection is used throughout backend controllers/services.
4	**Observer**	**4/5 - STRONG**	MutationObserver reacts to DOM changes and triggers translation.
5	**MVC / Layered Architecture**	**4/5 - STRONG**	Controllers, models, repositories/services, and React views are clearly separated.
6	**Provider / Context**	**3/5 - MODERATE**	React contexts centralize auth, language, and theme state.
7	**Service Layer**	**3/5 - MODERATE**	Reusable domain/system services such as AuditService and profile rules.
8	**State-based Workflow**	**2/5 - CONCEPTUAL**	Enums model states, but there are no separate GoF State classes.
9	**Spring Singleton Scope**	**2/5 - SUPPORTING**	Default Spring bean scope behaves like singleton; not a custom Singleton implementation.

Ranking meaning

Strongest / Strong: clearly implemented and safe to defend in a viva. **Moderate:** useful architectural pattern but less central to the Lab 06 task. **Conceptual / Supporting:** related pattern behavior exists, but it should not be claimed as a complete custom GoF implementation.

3. Strategy Pattern

5/5
STRONGEST - EXPLICIT GoF

Purpose: Encapsulate different payment algorithms behind one common interface. The payment controller no longer contains the detailed card and bank-transfer processing logic.

Why it fits Safari Lanka: A tourist can pay for the same confirmed booking using either Card or Bank Transfer. Both are payment behaviors, but their validation and completion rules are different.



PaymentStrategy.java - Strategy Interface

```
public interface PaymentStrategy {
    PaymentMethod supports();
    Payment process(PaymentStrategyCommand command) throws IOException;
}
```

PaymentContext.java - Context Class

```
@Service
public class PaymentContext {
    private final Map<PaymentMethod, PaymentStrategy> strategies;

    public Payment process(PaymentMethod method,
                           PaymentStrategyCommand command) throws IOException {
        PaymentStrategy strategy = strategies.get(method);
        if (strategy == null)
            throw new IllegalArgumentException("Unsupported payment method: " + method);
        return strategy.process(command);
    }
}
```

PaymentController.java - Client Delegation

```
public Payment payCard(...) throws IOException {
    Booking b = ownedPayableBooking(bookingId, a);
    return paymentContext.process(
        PaymentMethod.CARD,
        PaymentStrategyCommand.card(b, r, a.getName())
    );
}
```

4. Strategy Concrete Classes

5/5
CARD + BANK TRANSFER

Each concrete class owns the rules for one payment method. This is the key improvement: adding another method later, such as a wallet or gateway, can be done by creating another strategy instead of expanding the controller with more payment-specific logic.

CardPaymentStrategy.java

```
@Component
public class CardPaymentStrategy implements PaymentStrategy {
    @Override
    public PaymentMethod supports() { return PaymentMethod.CARD; }

    @Override
    public Payment process(PaymentStrategyCommand command) {
        PaymentRequest request = command.cardRequest();
        String digits = request.cardNumber().replace(" ", "");
        if (!digits.matches("\\d{16}"))
            throw new IllegalArgumentException("Card number must contain exactly 16 digits");

        validateExpiry(request.expiry());
        // create payment, mark PAID, update booking payment status, audit
        ...
    }
}
```

BankTransferPaymentStrategy.java

```
@Component
public class BankTransferPaymentStrategy implements PaymentStrategy {
    @Override
    public PaymentMethod supports() { return PaymentMethod.BANK_TRANSFER; }

    @Override
    public Payment process(PaymentStrategyCommand command) throws IOException {
        String reference = command.bankReference();
        MultipartFile slip = command.bankSlip();
        // validate reference + PDF, save slip
        // create PENDING payment for Finance review
        ...
    }
}
```

Behavior preserved

Card payments still become **PAID** immediately in the academic simulation. Bank transfers still become **PENDING** and require Finance approval. Tourists still cannot pay before Safari Operator confirmation, and resource assignment still requires a successful/approved payment.

5. Repository Pattern

5/5
STRONG

Purpose: Hide data-access details behind repository interfaces. Controllers and services work with domain-focused methods instead of writing SQL directly.

Where: `backend/src/main/java/com/safarilanka/repository/`

BookingRepository.java

```
public interface BookingRepository extends JpaRepository<Booking, Long> {
    List<Booking> findByTouristEmailOrderByCreatedAtDesc(String email);
    List<Booking> findBySafariPackageIdAndVisitDate(Long packageId, LocalDate date);
    Optional<Booking> findByBookingCode(String code);
}
```

Benefits in this project: less duplicated database code, readable queries, easier testing, separation between business logic and persistence, and consistent CRUD behavior across Booking, Payment, Vehicle, Permit, Review, Incident and other entities.

Viva sentence

“We use the Repository Pattern so controllers and services do not directly depend on SQL. Spring Data repositories provide a clean persistence abstraction for each entity.”

6. Dependency Injection

5/5
STRONG

Purpose: Objects receive their dependencies from Spring instead of constructing them manually with new. This reduces coupling and improves testability.

Constructor Injection in PaymentController

```
public PaymentController(
        PaymentRepository p,
        BookingRepository b,
        RefundRequestRepository r,
        AuditService a,
        PaymentContext paymentContext) {
    payments = p;
    bookings = b;
    refunds = r;
    audit = a;
    this.paymentContext = paymentContext;
}
```

Constructor Injection in a Concrete Strategy

```
public CardPaymentStrategy(
        PaymentRepository payments,
        BookingRepository bookings,
        AuditService audit) {
    this.payments = payments;
    this.bookings = bookings;
    this.audit = audit;
}
```

Why it matters

The Strategy implementation itself is also wired using Dependency Injection. Spring discovers CardPaymentStrategy and BankTransferPaymentStrategy and supplies them to PaymentContext as a list of PaymentStrategy implementations.

7. Observer Pattern

4/5
STRONG FRONTEND EXAMPLE

Purpose: React to changes automatically. Safari Lanka uses the browser MutationObserver in LanguageContext to observe DOM changes and translate newly inserted/changed text using the selected system language.

LanguageContext.jsx - Observer Behavior

```
const observer = new MutationObserver(mutations => {
  for (const m of mutations) {
    if (m.type === 'characterData')
      translateNode(m.target, language);
    else
      for (const node of m.addedNodes)
        translateNode(node, language);
  }
});

observer.observe(document.body, {
  subtree: true, childList: true, characterData: true
});
```

Why it is strong: the observer is explicitly registered, waits for changes, and runs a reaction when the observed subject changes. This is a practical Observer-style implementation in the frontend.

8. Provider / Context Pattern

3/5
MODERATE REACT ARCHITECTURE

Safari Lanka uses React Context providers for cross-application state such as authentication, system language, and light/dark theme. Components consume the shared state without passing it manually through every component level.

ThemeContext.jsx

```
export function ThemeProvider({children}) {
  const [theme, setTheme] = useState(...);
  const toggleTheme = () => setTheme(t => t === 'dark' ? 'light' : 'dark');
  return (
    <ThemeContext.Provider value={{theme, setTheme, toggleTheme}}>
      {children}
    </ThemeContext.Provider>
  );
}
```

9. MVC / Layered Architecture

4/5
STRONG ARCHITECTURE

Safari Lanka has a clear layered structure. Although the frontend is React rather than a classic server-rendered View, responsibilities are still separated into domain models, controllers/API endpoints, repositories/services, and frontend views/components.

Layer	Examples	Responsibility
View / UI	React pages, dashboards, components	User interaction and presentation
Controller	BookingController, PaymentController, PermitController	HTTP/API request orchestration
Service	AuditService, CurrentUserService, PaymentContext	Reusable application/domain logic
Repository	BookingRepository, PaymentRepository, VehicleRepository	Database access abstraction
Model	Booking, Payment, Permit, Vehicle, User	Persistent domain data

10. Service Layer

3/5
MODERATE

AuditService.java

```
@Service
public class AuditService {
    private final AuditLogRepository repo;

    public void log(String email, String action, String details) {
        AuditLog a = new AuditLog();
        a.setActorEmail(email);
        a.setAction(action);
        a.setDetails(details);
        repo.save(a);
    }
}
```

The service layer keeps reusable application operations outside controllers. The new PaymentContext also strengthens this layer by moving payment-method selection out of PaymentController.

11. State-based Workflow

2/5
CONCEPTUAL - NOT FULL GoF STATE

Safari Lanka models lifecycle states using enums such as BookingStatus, PaymentStatus, PermitStatus, AssignmentStatus and VehicleStatus.

State Enums

```
public enum BookingStatus {
    PENDING, CONFIRMED, REJECTED, CANCELLED, COMPLETED
}

public enum PaymentStatus {
    UNPAID, PENDING, PAID, REJECTED, FAILED, REFUNDED
}

public enum PermitStatus {
    PENDING, APPROVED, REJECTED, CHECKED_IN, COMPLETED, VOID
}
```

Important viva caution

Do not claim this as a full GoF State Pattern. The system has state-driven workflows, but behavior is still implemented with enums and conditional logic. A full State Pattern would normally have separate state classes such as PendingPaymentState, PaidPaymentState, etc.

12. Spring Singleton-Scoped Beans

2/5
SUPPORTING FRAMEWORK BEHAVIOR

Spring components such as `@Service`, `@Component` and `@RestController` are singleton-scoped by default unless another scope is configured. This provides singleton-like lifecycle behavior.

Important viva caution

This should be described as **Spring-managed singleton scope**, not as a custom Singleton Pattern implementation, because the project does not manually implement a private constructor plus static getInstance() method.

13. Exact Files Added for the Strategy Pattern

- `backend/src/main/java/com/safarilanka/service/payment/PaymentStrategy.java`
- `backend/src/main/java/com/safarilanka/service/payment/PaymentStrategyCommand.java`
- `backend/src/main/java/com/safarilanka/service/payment/PaymentContext.java`
- `backend/src/main/java/com/safarilanka/service/payment/CardPaymentStrategy.java`
- `backend/src/main/java/com/safarilanka/service/payment/BankTransferPaymentStrategy.java`
- `backend/src/main/java/com/safarilanka/controller/PaymentController.java (refactored to delegate)`
- `docs/DESIGN_PATTERN_STRATEGY_IMPLEMENTATION.txt`
- `docs/Safari_Lanka_Design_Patterns_Report.pdf`

14. Recommended Viva Order

Priority	What to present	One-sentence explanation
1	**Strategy**	Different payment methods implement one common PaymentStrategy and PaymentContext selects the correct one at runtime.
2	**Repository**	Repositories isolate persistence logic so controllers/services do not write SQL directly.
3	**Dependency Injection**	Spring injects repositories/services/strategies through constructors, reducing coupling.
4	**Observer**	MutationObserver watches DOM changes and automatically applies translation to updated UI content.

Best answer if asked “Which design pattern did your team implement?”

“We explicitly implemented the **Strategy Pattern** in the payment module. PaymentStrategy is the common interface, CardPaymentStrategy and BankTransferPaymentStrategy are concrete strategies, and PaymentContext selects the strategy at runtime. This follows the same structure described in our Lab 06.”

15. Final Assessment

The latest Safari Lanka build now has a **clear, explicit and defensible Strategy Pattern implementation**. Repository and Dependency Injection are also very strong backend patterns, while Observer gives the group a good frontend example. The report intentionally labels State and Singleton behavior cautiously so the group does not overclaim patterns that are not fully implemented as classic GoF structures.